WEAPONS ROBOTICS AND CONTROL ENGINEERING
UNITED STATES NAVAL ACADEMY

HOPPER INFORMATION SERIES

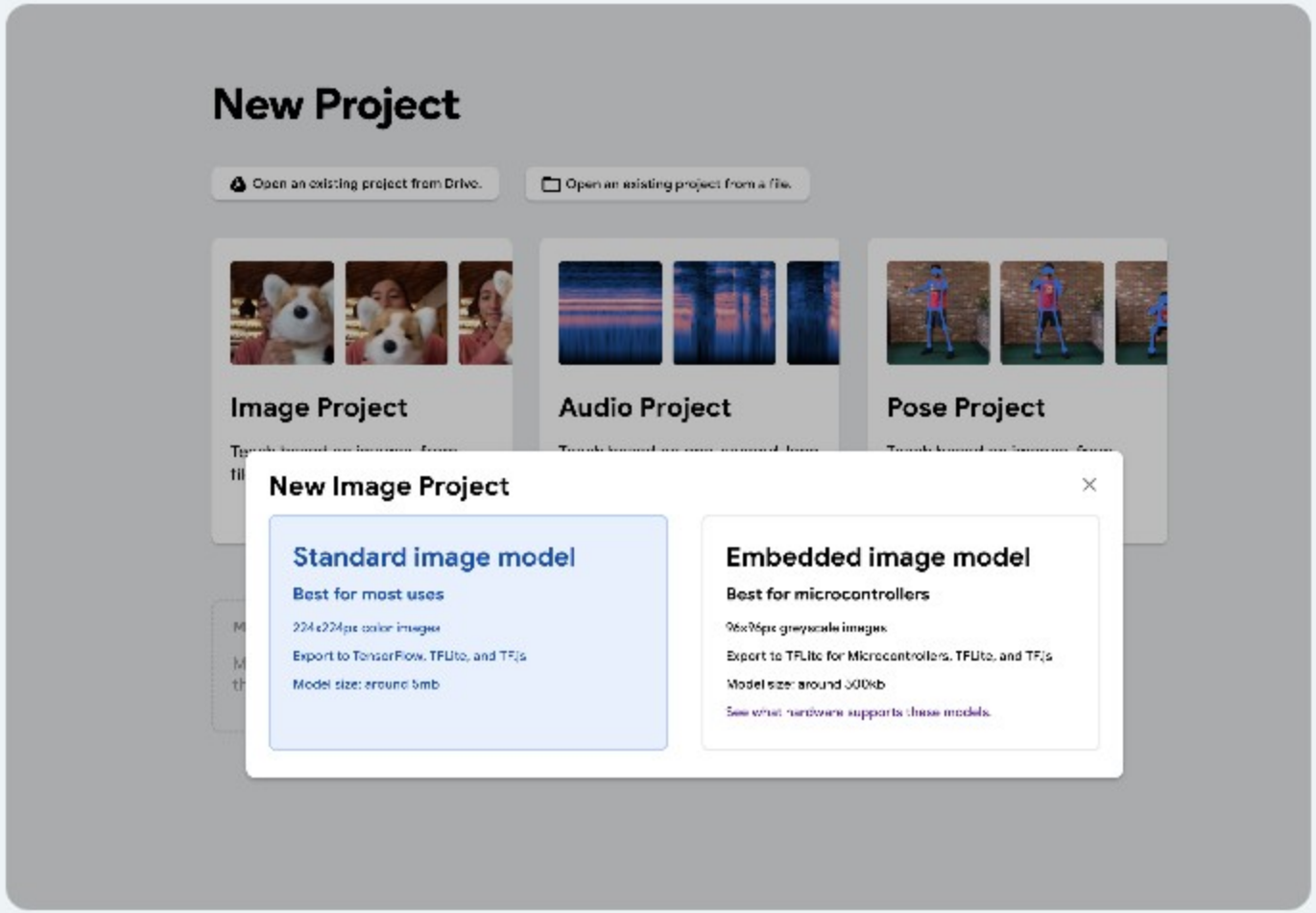
07 / CUSTOM IMAGE CLASSIFICATION

Build a Teachable Machine model

Train custom image classes in the browser, export TensorFlow.js files and load them into Hopper Studio.

COLLECT TRAIN EXPORT LOAD

Choose a standard image model



For Hopper Studio

- Select Image Project.
- Choose Standard image model.
- Export TensorFlow.js files after training.
- Keep model.json, weights.bin, and metadata.json together.

The embedded option targets microcontrollers; Hopper Studio loads the standard browser model.

Custom labels are the advantage—“other” is the guardrail

RED FLAG

Red fabric, multiple angles

BLUE FLAG

Blue fabric, multiple distances

SAFE PAD

Target pad under flight lighting

OTHER / BACKGROUND

Floor, walls, people, blur, empty room

Why the “other” class matters

- A classifier must distribute probability across the labels it knows.
- Without negative/background examples, an irrelevant frame can be forced into a mission label.
- Make “other” diverse: empty views, different floors, students, walls, shadows, motion blur, and anything you do not want to trigger.

Choose labels that answer one mission question—and collect the world outside those labels.

NEURAL NETWORK

What goes into—and comes out of—the custom model

01

Camera frame

One current image from the real or simulated view.

02

Resize + crop

Standard image models work from a square image tensor.

03

Features

Pretrained MobileNet-like layers extract visual features.

04

Custom head

Transfer learning maps those features to your class names.

05

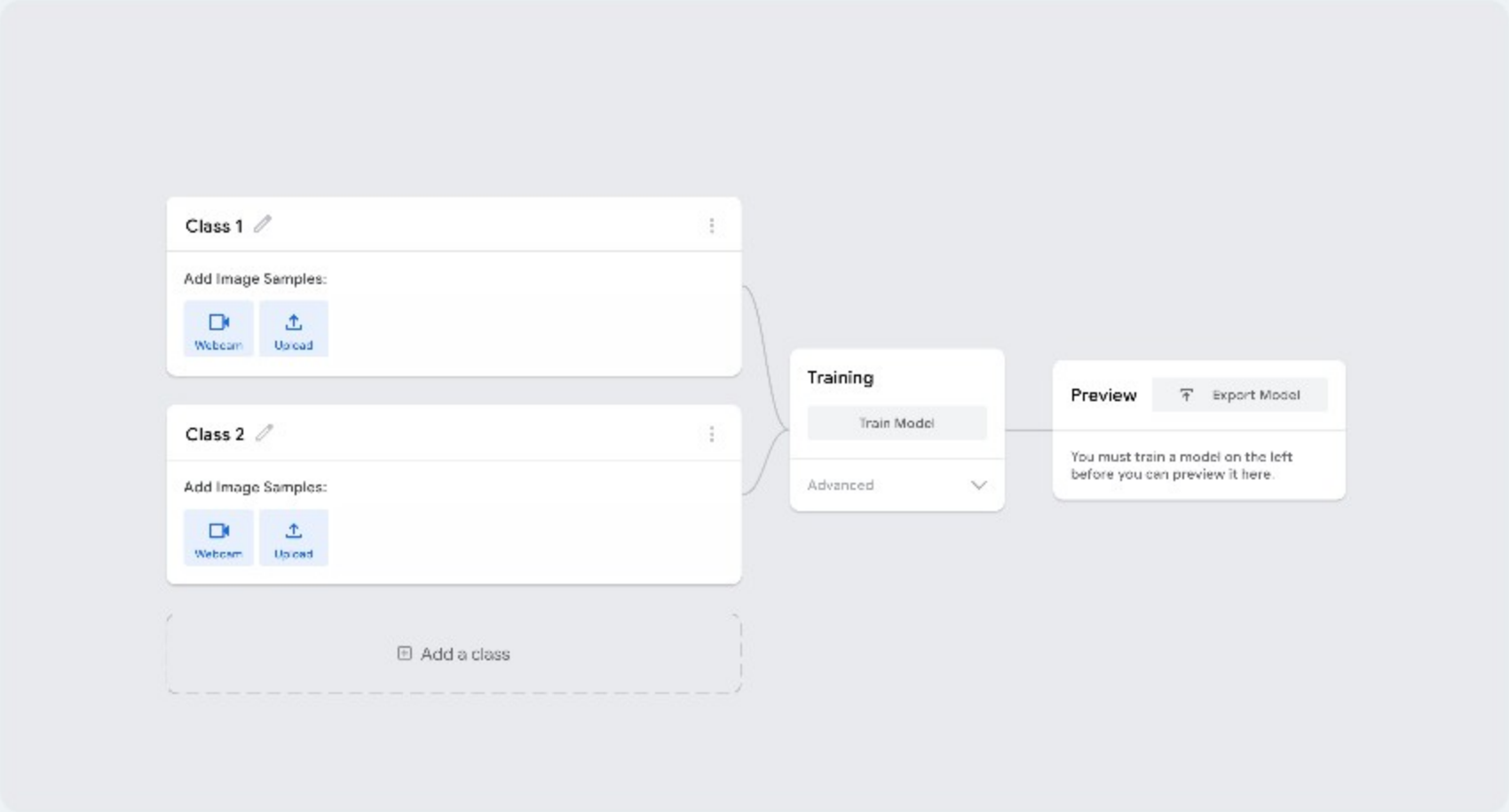
Probabilities

One score per label; highest score becomes the current class.

EXAMPLE OUTPUT

Whole-frame classification: no x/y coordinates and no bounding boxes.

Use the training screen as an experiment dashboard



1 · COLLECT BALANCED CLASSES

2 · TRAIN

3 · PREVIEW UNSEEN VIEWS

THE REQUIRED TRIO

Export the standard TensorFlow.js files

1 Export Model

Choose TensorFlow.js.

→

2 Download

Download my model.

→

3 Unzip

Keep the three files together.

`hopper-model/`

`model.json`	network structure
`weights.bin`	learned parameters
`metadata.json`	labels + project metadata

Not supported: .tflite, embedded export or only one of the three files.

LOCAL APP WORKFLOW

Load the model into Hopper Studio

1 Connect

Use simulator or readable camera feed.

2 Open Vision

Find Teachable Machine in Object Detector.

3 Load Model

Select all three files together.

4 Confirm

Class labels appear in the panel.

5 Scan

Test once before flight code.

IMPORTANT: THE MODEL IS NOT FLASHED INTO THE AIRCRAFT

Hopper Studio loads the files into browser/app memory and runs inference on this computer.

Reload after refresh. The model files are not persisted or uploaded to a cloud service.

MODEL.JSON WEIGHTS.BIN METADATA.JSON

WHOLE-FRAME PROBABILITY

Use the custom class in blocks or JavaScript

custom model sees red pad at 75% confidence

CUSTOM CLASS CHECK

```
const redPad = await
  vision.seesCustomLabel("red pad", 0.75);
if (redPad) {
  await drone.hover();
  await drone.land();
}
```

WHAT YOU GET

- className + probability for every class.
- Exact class-name match, case-insensitive.
- No bounding box, x/y or multiple-object location.

FRESH SCAN

- seesCustomLabel captures a new frame.
- Confidence in JavaScript is 0..1.
- Scanning includes animation + inference time.
- Do not use it as a high-rate control sensor.

ITERATIVE VALIDATION

Treat every failure as new training data

1. **Test**
   Run with propellers removed or simulator first.
2. **Record**
   Save false positives and misses.
3. **Label**
   Add them to the correct class.
4. **Balance**
   Keep class counts and variation similar.
5. **Retrain**
   Export and load a new version.

FLIGHT ACCEPTANCE TEST

- Target class succeeds across distance/rotation.
- Negative class rejects empty floor.
- No single lighting condition dominates.
- Threshold chosen from measured errors.

SOURCES AND FURTHER READING

Google Teachable Machine
https://teachablemachine.withgoogle.com/train

TensorFlow.js transfer learning overview
https://codelabs.developers.google.com/tensorflowjs-transfer-learning-teachable-machine

Hopper Studio custom-model implementation
lib/vision.ts and README.md

ACCEPTANCE TEST

A custom model is only ready after an honest field test

PASS CRITERIA

- Correct label across expected distance, angle, and light.
- Low false-trigger rate on the “other” class.
- Stable predictions during short motion blur and camera changes.
- Works on held-out examples that were never used in training.

IF IT FAILS

- Record the failure view.
- Label it correctly—often as other/background.
- Rebalance classes so one does not dominate.
- Retrain, export all three files, and repeat the same test.

Custom labels are powerful because they match your mission. The cost is owning the data, the negative class, and the validation.